



Home Work

DevOps

HW_L3_03_Ansibl

e

۱. فایل تمرین را در پنل خود آپلود کنید.

۲. title فایل تمرین به صورت (نام تمرین+نام و نام خانوادگی) به انگلیسی باشد.

نمونه: AmirMohammadMohammadi_ansible_03

۳. در صورتی که سوال و یا ابهامی دارید در گروه چت تلگرامی بپرسید.

تمرین Ansible

مجموع امتیاز: سی نمره

تمام سناریوها روی یک سیستم Ubuntu 22.04 یا WSL/VM لوکال قابل اجرا هستند

برای هر سوال فایل و کانفیگ نمونه داده شده تا دقیقاً همان را بسازید

تحويل یک پوشه کامل شامل پوشه‌های هر سناریو، inventory، کانفیگ‌ها و خروجی دستورها

پیش‌نیاز: دسترسی به حداقل یک سیستم remote می‌تواند VM ، WSL، یا container باشد

ساختار پیشنهادی پوشه کار:

ansible_homework/

├── 01_ansible_install/

├── 02_ansible_basics/

├── 03_ansible_concepts/

├── 04_ansible_components/

└── 05_ansible_adhoc/

مجموعه سوالات اول: نصب و آماده‌سازی — Ansible شش نمره

سوال ۱.۱ — نصب — Ansible سه نمره

آنچه می‌خواهم: نصب Ansible و بررسی نسخه آن

• فایل اسکریپت ایجاد کنید install_ansible.sh در 01_ansible_install/ :

• نمونه محتوای اسکریپت:

```
#!/bin/bash
```

```
set -e
```

#نصب Python و pip اگر نصب نیست

```
sudo apt update
```

```
sudo apt install -y python3 python3-pip python3-venv
```

#نصب Ansible با pip

```
pip3 install --user ansible
```

#اضافه کردن pip bin به PATH

```
export PATH=$PATH:~/local/bin
```

#بررسی نسخه

```
ansible --version > ansible_version.txt
```

#بررسی modules نصب شده

```
ansible-doc -l | head -30 >> ansible_version.txt
```

#بررسی تعداد کل modules

```
echo "Total modules: $(ansible-doc -l | wc -l)" >> ansible_version.txt
```

خروجی دستورهای زیر را در 01_ansible_install/ansible_status.txt ذخیره کنید:

```
which ansible
```

```
ansible --version
```

```
ansible-config dump | head -30
```

```
python3 --version
```

```
pip3 show ansible
```

یک فایل 01_ansible_install/installation_notes.md بسازید و به این سوالات پاسخ دهید:

۱. چرا از `pip install --user` استفاده می‌کنیم؟

۲. تفاوت نصب با `apt` و `pip` چیست؟

۳. کدام روش برای `production` بهتر است و چرا؟

Hints:

- می‌توانید از `apt install ansible` استفاده کنید نسخه قدیمی‌تر
- برای `production`، استفاده از `pip` توصیه می‌شود
- `ansible-config dump` تمام تنظیمات Ansible را نشان می‌دهد

سوال ۱.۲ — پیکربندی اولیه — Ansible سه نمره

آنچه می‌خواهیم: ایجاد ساختار اولیه و پیکربندی Ansible

- پوشه 01_ansible_install/ansible_config بسازید

- فایل ansible.cfg ایجاد کنید:

```
[defaults]
```

```
inventory = ./inventory
```

```
host_key_checking = False
```

```
remote_user = ubuntu
```

```
private_key_file = ~/.ssh/id_rsa
```

```
retry_files_enabled = False
```

```
stdout_callback = yaml
```

```
deprecation_warnings = True
```

```
[privilege_escalation]
```

```
become = True
```

```
become_method = sudo
```

```
become_user = root
```

```
become_ask_pass = False
```

```
[inventory]
```

```
enable_plugins = host_list, script, auto, yaml, ini, toml
```

- فایل inventory ساده ایجاد کنید:

```
[local]
```

```
localhost ansible_connection=local
```

```
ansible_python_interpreter=/usr/bin/python3
```

```
[webservers]
```

```
# web1 ansible_host=192.168.1.10 ansible_user=ubuntu
```

```
# web2 ansible_host=192.168.1.11 ansible_user=ubuntu
```

```
[dbservers]
```

```
# db1 ansible_host=192.168.1.12 ansible_user=ubuntu
```

```
[all:vars]
```

```
ansible_python_interpreter=/usr/bin/python3
```

- دستورهای زیر را در 01_test_connection/ansible_install/ ذخیره و اجرا کنید:

```
#!/bin/bash
```

```
cd ansible_config
```

#تست اتصال به localhost

```
echo "=== Connection Test ===" > connection_test.txt
```

```
ansible localhost -m ping >> connection_test.txt
```

#مشاهده inventory به صورت JSON

```
echo -e "\n=== Inventory List ===" >> connection_test.txt
```

```
ansible-inventory --list >> connection_test.txt
```

#مشاهده hosts

```
echo -e "\n=== All Hosts ===" >> connection_test.txt
```

```
ansible all --list-hosts >> connection_test.txt
```

#مشاهده variables برای localhost

```
echo -e "\n=== Host Variables ===" >> connection_test.txt
```

ansible-inventory --host localhost >> connection_test.txt

- یک فایل 01_ansible_install/config_explanation.md بسازید و هر directive در ansible.cfg را توضیح دهید

Hints:

- False = host_key_checking فقط برای development است
- در production ، از SSH keys استفاده کنید
- localhost=ansible_connection برای اجرای دستورات روی localhost

مجموعه سوالات دوم: مبانی — ansible-1-1 Ansible نه نمره

سوال ۲.۱ — درک کامل Ansible و معماری آن — پنج نمره

آنچه می‌خواهم: توضیح کامل Ansible و معماری آن

- فایل 02_ansible_basics/ansible_complete_explanation.md بسازید و به این سوالات به صورت کامل پاسخ دهید:

۱. Ansible چیست و چرا استفاده می‌شود؟

○ توضیح دهید که Ansible یک automation platform است

○ تاریخچه مختصر Ansible توسعه توسط Michael DeHaan در ۲۰۱۲

○ مزایای اصلی استفاده از: Ansible

▪ **Idempotency:** اجرای چندباره نتیجه یکسان

▪ **Agentless:** نیازی به agent روی target machines نیست

▪ **Simplicity:** استفاده از Python و YAML

▪ **Powerful:** پشتیبانی از هزاران module

○ تفاوت Ansible با ابزارهای دیگر:

▪ **Puppet vs Puppet:** Pull model استفاده می‌کند، Ansible از Push

▪ **Chef vs Chef:** پیچیده‌تر است، Ansible ساده‌تر

▪ **SaltStack vs SaltStack:** سریع‌تر اما پیچیده‌تر

۲. معماری: Ansible

○ **Control Node:** جایی که Ansible نصب است و دستورات از آنجا اجرا می‌شوند

○ **Managed Nodes:** سیستم‌های target که مدیریت می‌شوند

○ **Inventory:** لیست managed nodes

○ **Modules:** واحدهای code که tasks را انجام می‌دهند

○ **Plugins:** افزونه‌ها برای گسترش قابلیت‌ها

- نمودار معماری را با ASCII art بکشید

۳. Agentless Architecture:

- توضیح دهید که Ansible نیازی به agent ندارد
- چگونه Ansible با remote machines ارتباط برقرار می‌کند؟ SSH برای Linux ، WinRM برای Windows

- مزایای agentless:

- نصب و نگهداری ساده‌تر

- امنیت بیشتر نیازی به port باز نیست

- شروع سریع‌تر

- معایب agentless:

- نیاز به SSH/WinRM

- نیاز به Python روی target برای اکثر modules

۴. Push vs Pull Model:

- **Push Model Ansible:** Control node دستورات را به managed nodes می‌فرستد
- **Pull Model Puppet:** Managed nodes خودشان به server متصل می‌شوند

○ مزایا و معایب هر کدام

○ چرا Ansible از Push model استفاده می کند؟

- یک فایل 02_ansible_basics/ansible_architecture.txt بسازید و نمودار کامل معماری Ansible را با متن توضیح دهید

سوال ۲.۲ — کار با Inventory چهار نمره

آنچه می خواهیم: درک کامل Inventory و انواع آن

- فایل inventory در 02_ansible_basics/ بسازید:

```
[webservers]
```

```
web1 ansible_host=localhost ansible_connection=local
```

```
web2 ansible_host=localhost ansible_connection=local
```

```
# web3 ansible_host=192.168.1.10 ansible_user=ubuntu
```

```
[dbservers]
```

```
db1 ansible_host=localhost ansible_connection=local
```

```
# db2 ansible_host=192.168.1.11 ansible_user=ubuntu
```

```
[appservers]
```

```
app1 ansible_host=localhost ansible_connection=local
```

```
[production:children]
```

```
webservers
```

```
dbservers
```

```
[all:vars]
```

```
ansible_python_interpreter=/usr/bin/python3
```

```
ansible_user=ubuntu
```

```
[webservers:vars]
```

```
http_port=80
```

```
max_clients=200
```

```
[dbservers:vars]
```

```
db_port=3306
```

- دستورهای زیر را در `02_ansible_basics/inventory_operations.sh` ذخیره و اجرا کنید:

```
#!/bin/bash
```

#مشاهده inventory به صورت JSON

```
echo "=== Inventory as JSON ===" > inventory_operations.txt
```

```
ansible-inventory -i inventory --list >> inventory_operations.txt
```

#مشاهده inventory به صورت YAML

```
echo -e "\n=== Inventory as YAML ===" >> inventory_operations.txt
```

```
ansible-inventory -i inventory --yaml >> inventory_operations.txt
```

#مشاهده hosts در گروه webservers

```
echo -e "\n=== Webservers Group ===" >> inventory_operations.txt
```

```
ansible-inventory -i inventory --list --host web1 >> inventory_operations.txt
```

#مشاهده همه hosts

```
echo -e "\n=== All Hosts ===" >> inventory_operations.txt
```

```
ansible all -i inventory --list-hosts >> inventory_operations.txt
```

#مشاهده *hosts* در گروه *production parent group*

```
echo -e "\n=== Production Group Children ===" >>  
inventory_operations.txt
```

```
ansible production -i inventory --list-hosts >> inventory_operations.txt
```

#مشاهده *variables* برای یک *host*

```
echo -e "\n=== Variables for web1 ===" >> inventory_operations.txt
```

```
ansible-inventory -i inventory --host web1 >> inventory_operations.txt
```

#تست *ping* به همه *hosts*

```
echo -e "\n=== Ping All Hosts ===" >> inventory_operations.txt
```

```
ansible all -i inventory -m ping >> inventory_operations.txt
```

- یک فایل `02_ansible_basics/inventory_explanation.md` بسازید و به این سوالات پاسخ دهید:

۱. Inventory چیست و چه نقشی دارد؟

۲. تفاوت `INI format` و `YAML format` برای `inventory` چیست؟

۳. `Groups` و `Parent Groups` چیستند؟

۴. Variables در inventory چگونه تعریف می‌شوند؟

۵. `ansible_connection=local` چه کاری انجام می‌دهد؟

Hints:

- `ansible-inventory --list` ساختار کامل inventory را نشان می‌دهد
- می‌توانید از `YAML format` هم برای inventory استفاده کنید
- `Groups` برای سازمان‌دهی `hosts` استفاده می‌شوند
- `Parent groups` با `children`: تعریف می‌شوند

مجموعه سوالات سوم: مفاهیم پیشرفته — `Ansible ansible-1-2` نه نمره

سوال ۳.۱ `Idempotency` — و — `State Management` پنج نمره

آنچه می‌خواهم: درک کامل `Idempotency` و `State Management`

- فایل `03_ansible_concepts/idempotency_explanation.md` بسازید و به این سوالات به صورت کامل پاسخ دهید:

۱. `Idempotency` چیست؟

○ تعریف دقیق: اجرای چندباره یک `operation` نتیجه یکسان دارد

○ چرا `idempotency` مهم است؟

- امنیت: اجرای دوباره خطا ایجاد نمی‌کند
- قابلیت اطمینان: می‌توانید playbook را چندبار اجرا کنید
- Debugging راحت‌تر می‌توانید مشکل را پیدا کنید
- مثال‌های عملی:

- نصب package: اگر نصب باشد، دوباره نصب نمی‌شود
- ایجاد file: اگر وجود داشته باشد، دوباره ایجاد نمی‌شود
- Start service: اگر running باشد، دوباره start نمی‌شود

۲. State Management:

- Ansible از state-based approach استفاده می‌کند
- شما state مورد نظر را تعریف می‌کنید، نه actions
- مثال state: present: به جای install package
- مزایای state management:

▪ Idempotency خودکار

▪ خوانایی بهتر

▪ نگهداری آسان‌تر

۳. چگونه Ansible Idempotency را تضمین می کند؟

- Modules قبل از تغییر، state فعلی را بررسی می کنند
 - فقط در صورت نیاز تغییر ایجاد می کنند
 - changed: false یعنی تغییری ایجاد نشده
 - changed: true یعنی تغییری ایجاد شده
- دستورهای زیر را در 03_ansible_concepts/idempotency_demo.sh ذخیره و اجرا کنید:

```
#!/bin/bash
```

```
echo "=== First Run should create ===" > idempotency_demo.txt
```

```
ansible localhost -m file -a "path=/tmp/ansible_idempotency state=directory"  
>> idempotency_demo.txt
```

```
echo -e "\n=== Second Run should NOT create, changed=false ===" >>  
idempotency_demo.txt
```

```
ansible localhost -m file -a "path=/tmp/ansible_idempotency state=directory"  
>> idempotency_demo.txt
```

```
echo -e "\n=== Third Run should NOT create, changed=false ===" >>  
idempotency_demo.txt
```

```
ansible localhost -m file -a "path=/tmp/ansible_idempotency state=directory"
>> idempotency_demo.txt
```

#مشاهده خروجی - باید ببینید که فقط اولین بار *changed: true* است

سوال ۳.۲ Modules — و — Documentation چهار نمره

آنچه می‌خواهم: درک کامل Modules و نحوه استفاده از Documentation

- فایل 03_ansible_concepts/modules_explanation.md بسازید و به این سوالات پاسخ دهید:

۱. Module چیست؟

- Module یک unit از code است که یک task خاص را انجام می‌دهد
- Modules در کجا اجرا می‌شوند؟ روی target machines
- Modules چگونه به target machines منتقل می‌شوند؟ از طریق Ansible
- Built-in modules vs Custom modules

۲. انواع: Modules

- **System Modules:** apt, yum, service, user, group
- **File Modules:** copy, template, file, lineinfile, blockinfile

- **Command Modules:** `command`, `shell`, `script`, `raw`
- **Cloud Modules:** `ec2`, `gce`, `azure_rm`
- **Network Modules:** `uri`, `get_url`, `slurp`
- **Database Modules:** `mysql_db`, `postgresql_db`

○ برای هر دسته، دو مثال و کاربرد آن‌ها را بنویسید

۳. تفاوت `raw`، `command`، `shell`

- `command`: اجرای دستور بدون `shell` امن‌تر، `idempotent` نیست
- `shell`: اجرای دستور با `shell` می‌تواند از `pipes` و `redirects` استفاده کند
- `raw`: اجرای دستور بدون `module` برای `systems` بدون `Python`
- چه زمانی از هر کدام استفاده کنیم؟
- دستورهای زیر را در `03_ansible_concepts/modules_documentation.sh` ذخیره و اجرا کنید:

```
#!/bin/bash
```

لیست همه modules

```
echo "=== All Modules ===" > modules_doc.txt
```

```
ansible-doc -l | head -50 >> modules_doc.txt
```

apt module برای # Documentation

```
echo -e "\n=== apt Module Documentation ===" >> modules_doc.txt
```

```
ansible-doc apt | head -100 >> modules_doc.txt
```

file module برای # Documentation

```
echo -e "\n=== file Module Documentation ===" >> modules_doc.txt
```

```
ansible-doc file | head -100 >> modules_doc.txt
```

copy module برای # Documentation

```
echo -e "\n=== copy Module Documentation ===" >> modules_doc.txt
```

```
ansible-doc copy | head -100 >> modules_doc.txt
```

ping module برای # Examples

```
echo -e "\n=== ping Module Examples ===" >> modules_doc.txt
```

```
ansible-doc -t module ping | grep -A 20 "EXAMPLES" >> modules_doc.txt
```

- یک فایل `03_ansible_concepts/module_usage_guide.md` بسازید و توضیح دهید:

۱. چگونه از `ansible-doc` استفاده کنیم؟

۲. چگونه `examples` را پیدا کنیم؟

۳. چگونه `parameters` را درک کنیم؟

مجموعه سوالات چهارم Components: اصلی — Ansible ansible-1-3 چهار نمره

سوال ۴.۱ Facts — و — Variables دو نمره

آنچه می‌خواهم: درک کامل `Facts` و `Variables`

- فایل `04_ansible_components/facts_variables_explanation.md` بسازید و به این

سوالات پاسخ دهید:

۱. `Facts` چیست؟

○ `Facts` اطلاعات جمع‌آوری شده از `target machines` هستند

○ `gather_facts: yes` چه اطلاعاتی جمع می‌کند؟

- System info: `hostname`, `OS`, `architecture`
- Network info: `IP addresses`, `interfaces`
- Hardware info: `CPU`, `memory`, `disks`

- چگونه می‌توانیم facts را مشاهده کنیم؟ (ansible -m setup)
- ansible_facts vs ansible_* variables

۲. Ansible: Variables در

- چگونه تعریف می‌شوند؟
 - در: playbook: vars
 - در: inventory: [group:vars]
 - در: command line: -e "var=value"
 - در: files: vars_files
- Precedence اولویت variables چیست؟
- چگونه از variables استفاده می‌کنیم؟ ({{ variable_name }})
- Jinja2 templating چیست؟
- دستورهای زیر را در 04_ansible_components/facts_demo.sh ذخیره و اجرا کنید:

```
#!/bin/bash
```

```
# Gather all facts
```

```
echo "=== All Facts ===" > facts_demo.txt
```

```
ansible localhost -m setup >> facts_demo.txt
```

```
# Filter specific facts
```

```
echo -e "\n=== Distribution Facts ===" >> facts_demo.txt
```

```
ansible localhost -m setup -a "filter=ansible_distribution*" >> facts_demo.txt
```

```
echo -e "\n=== Network Facts ===" >> facts_demo.txt
```

```
ansible localhost -m setup -a "filter=ansible_default_ipv4" >> facts_demo.txt
```

```
echo -e "\n=== Memory Facts ===" >> facts_demo.txt
```

```
ansible localhost -m setup -a "filter=ansible_memtotal_mb" >> facts_demo.txt
```

```
# Save facts to file
```

```
ansible localhost -m setup > facts_all.json
```

سوال ۴.۲ Configuration — و — Settings دو نمره

آنچه می‌خواهم: درک Configuration Management

- فایل 04_ansible_components/configuration_explanation.md بسازید و به این

سوالات پاسخ دهید:

۱. ansible.cfg چیست؟

○ فایل پیکربندی اصلی Ansible

○ اولویت فایل های: ansible.cfg

1. ANSIBLE_CONFIG environment variable

2. ansible.cfg در current directory

3. ~/.ansible.cfg

4. /etc/ansible/ansible.cfg

○ مهم ترین sections:

▪ [defaults]: تنظیمات پیش فرض

▪ [privilege_escalation]: تنظیمات sudo/su

▪ [inventory]: تنظیمات inventory

۲. مهم ترین Configuration Options:

○ inventory: مسیر file inventory

○ host_key_checking: بررسی SSH host keys

○ remote_user: کاربر پیش فرض برای اتصال

○ become: استفاده از privilege escalation

○ gathering: جمع آوری facts

- دستورهای زیر را در 04_ansible_components/config_demo.sh ذخیره و اجرا کنید:

```
#!/bin/bash
```

#مشاهده تنظیمات فعلی

```
echo "=== Current Configuration ===" > config_demo.txt
```

```
ansible-config dump >> config_demo.txt
```

#مشاهده تنظیمات برای یک option خاص

```
echo -e "\n=== Inventory Setting ===" >> config_demo.txt
```

```
ansible-config dump | grep inventory >> config_demo.txt
```

#مشاهده تمام تنظیمات مربوط به SSH

```
echo -e "\n=== SSH Settings ===" >> config_demo.txt
```

```
ansible-config dump | grep -i ssh >> config_demo.txt
```

#مشاهده تنظیمات privilege escalation

```
echo -e "\n=== Privilege Escalation ===" >> config_demo.txt
```

```
ansible-config dump | grep become >> config_demo.txt
```

مجموعه سوالات پنجم — Ad-Hoc Commands: یک نمره

سوال ۵.۱ — استفاده از Ad-Hoc Commands یک نمره

آنچه می‌خواهم: یادگیری کامل Ad-Hoc Commands

- فایل 05_ansible_adhoc/adhoc_explanation.md بسازید و به این سوالات پاسخ دهید:

۱. Ad-Hoc Command چیست؟

- دستورات سریع بدون نیاز به playbook
- چه زمانی از ad-hoc استفاده می‌کنیم؟
 - Tasks سریع و یکباره
 - Testing و troubleshooting
 - Quick changes
- تفاوت ad-hoc با playbook چیست؟
 - Ad-hoc: سریع، ساده، یک task
 - Playbook: پیچیده، قابل استفاده مجدد، چند task

۲. ساختار: Ad-Hoc Command

3. `ansible <hosts> -m <module> -a "<arguments>" [options]`

○ `<hosts>`: target hosts all, group, یا host خاص

○ `<module>`: module -m برای استفاده

○ `<arguments>`: arguments -a برای module

○ Options: -i inventory, -u user, --become sudo

• دستورهای زیر را در `05_ansible_adhoc/adhoc_commands.sh` ذخیره و اجرا کنید:

```
#!/bin/bash
```

```
# Ping test
```

```
echo "=== Ping Test ===" > adhoc_output.txt
```

```
ansible localhost -m ping >> adhoc_output.txt
```

```
# Gather facts
```

```
echo -e "\n=== Gather Facts ===" >> adhoc_output.txt
```

```
ansible localhost -m setup -a "filter=ansible_distribution*" >>  
adhoc_output.txt
```

File operations

```
echo -e "\n=== File Operations ===" >> adhoc_output.txt
```

```
ansible localhost -m file -a "path=/tmp/ansible_test state=directory  
mode=0755" >> adhoc_output.txt
```

```
ansible localhost -m stat -a "path=/tmp/ansible_test" >> adhoc_output.txt
```

Command execution

```
echo -e "\n=== Command Execution ===" >> adhoc_output.txt
```

```
ansible localhost -m command -a "uptime" >> adhoc_output.txt
```

```
ansible localhost -m shell -a "echo $HOME && whoami" >>  
adhoc_output.txt
```

Copy file

```
echo -e "\n=== Copy File ===" >> adhoc_output.txt
```

```
echo "Test content from ad-hoc" > /tmp/test_source.txt
```

```
ansible localhost -m copy -a "src=/tmp/test_source.txt  
dest=/tmp/test_dest.txt" >> adhoc_output.txt
```

Get URL

```
echo -e "\n=== Get URL ===" >> adhoc_output.txt
```

```
ansible localhost -m get_url -a "url=https://www.google.com  
dest=/tmp/google.html" >> adhoc_output.txt
```

Package management with become - skip if no sudo

```
echo -e "\n=== Package Management ===" >> adhoc_output.txt
```

```
ansible localhost -m apt -a "name=curl state=present update_cache=yes" --  
become >> adhoc_output.txt 2>&1 || echo "Skipped no sudo access" >>  
adhoc_output.txt
```

- خروجی فایل adhoc_output.txt را در پوشه تحویل قرار دهید
- یک فایل 05_ansible_adhoc/adhoc_best_practices.md بسازید و توضیح دهید:

۱. چه زمانی از ad-hoc استفاده کنیم؟

۲. چه زمانی باید playbook بنویسیم؟

۳. Best practices برای ad-hoc commands

Hints:

- Ad-hoc commands برای tasks سریع و یکباره مناسب هستند
- tasks پیچیده یا قابل استفاده مجدد، از playbook استفاده کنید

- become--برای اجرای دستورات با sudo
- i-برای مشخص کردن inventory file

تحويل نهایی و فایل خروجی — یک نمره

- یک فایل README.md در ریشه پوشه تمرین قرار دهید و شامل موارد زیر باشد:
 ۱. توضیح کوتاه پروژه
 ۲. نحوه راه اندازی نصب Ansible و پیکربندی
 ۳. ساختار فایل ها و نقش هر بخش
 ۴. نحوه اجرای ad-hoc commands
 ۵. خلاصه مفاهیم یاد گرفته شده
- خروجی دستور tree یا لیست find را در فایل final_structure.txt ذخیره کنید
- یک فایل ansible_commands_history.txt شامل تمام دستورات Ansible که اجرا کرده اید

معیار نمره دهی خلاصه

- نصب و پیکربندی صحیح Ansible: بیست درصد

- درک کامل مفاهیم و توضیحات: 1-1, 1-2, 1-3 ansible-1-1, 1-2, 1-3 چهل درصد
 - کار با inventory و modules: بیست درصد
 - استفاده صحیح از ad-hoc commands: ده درصد
 - رعایت دقیق مسیرها و نام فایل‌ها: پنج درصد
 - توضیحات کوتاه و مرتب در فایل‌های متنی: پنج درصد
- مهلت تحویل: جمعه